

EXAM NOTES

P1: Introduction to Processes

Operating Systems — Beginner-Friendly Reference

⚡ QUICK REFERENCE — `fork()` Return Values

Who is running?	<code>fork()</code> returns...
Parent process	PID of the child (a positive integer, e.g. 1234)
Child process	0 (zero — exactly zero)
Error occurred	-1 (minus one) — no child was created

📌 TRICK

To tell who you are after `fork()`: if `(pid > 0)` → you are the parent. if `(pid == 0)` → you are the child. if `(pid < 0)` → `fork()` failed.

1. WHAT IS A PROCESS?

A process is an instance of a program that is currently being executed by the CPU.

KEY DISTINCTION

A program is a static file sitting on disk (e.g., `/usr/bin/ls`). A process is that program while it is **RUNNING** in memory. Same program → many processes (e.g., two terminal windows running `"ls"` at once).

Why Processes Matter

- Core concept of every Operating System
- Allows many programs to run 'at the same time'
- OS tracks and manages each process independently

Single-core vs Multi-core

CPU Type	How it handles multiple processes
Single-core	Time-sharing: switches between processes so fast it feels simultaneous
Multi-core	True parallelism: runs processes at the same time on different cores, PLUS time-sharing within each core

What the OS Stores for Each Process (PCB)

The OS keeps a process table. Each row is a Process Control Block (PCB) — a snapshot of one process.

PCB Field	What it stores / why it matters
Process ID (PID)	Unique number identifying the process
Program Counter	Address of the next instruction to run
CPU Registers	Saved register values when process is paused
Memory Maps	Where in RAM the process lives
Open Files	List of files the process has open
State	Running, ready, blocked, etc.

EXAM

When a process is suspended (paused), **ALL** its state is saved in the PCB. When it resumes, the OS restores the PCB so execution continues exactly where it left off.

Paged Virtual Memory (How Processes Share RAM)

- Each process has its own virtual address space (pretends it owns all of RAM)
- A page table maps virtual addresses → real physical addresses
- Physical memory is shared among all processes — but they cannot see each other's pages
- Pages not in RAM are stored on disk and loaded on demand



ANALOGY

Virtual memory is like each apartment in a building having its own numbered rooms (virtual addresses). The building manager (OS) knows which physical room corresponds to each apartment's room number.

2. PROCESS LIFECYCLE (CREATION & TERMINATION)

Events That Create a Process

- **System boots up (init / systemd creates first processes)**
- User runs a program from the shell (e.g., typing 'ls')
- A shell script runs — each command in the script may create a process
- A running process explicitly creates a child process via `fork()`

Events That End a Process

Termination Cause	Description
Normal exit	Program finishes and calls <code>exit(0)</code> — success
Error exit	Program calls <code>exit(non-zero)</code> — signals failure
Fatal error	Crash, e.g., segmentation fault (bad memory access)
Killed by user	User sends Ctrl-C or uses the kill command
Killed by OS	OS terminates misbehaving process

Programmer's View: Birth, Life, Death

- **Birth** — A process is born by being cloned from its parent using `fork()`
- **Life** — The child can spawn its own children; it can also replace itself with a new program (`exec`)
- **Death** — The process calls `exit()` or is killed; its parent should collect its exit status



CRITICAL

Every process has a parent. There are NO parentless processes (except `init/PID 1`). When you call `fork()`, the calling process is the parent and the new one is the child.

3. fork() — CLONING A PROCESS

fork() is the ONLY way to create a new process in Unix/Linux. It creates an exact copy (clone) of the calling process.

fork() Signature

```
#include <unistd.h>

pid_t  fork(void);

// pid_t is just an integer type for process IDs
```

What fork() Does Step-by-Step

- **1. OS makes a complete copy of the calling process (parent)**
- 2. Both parent and child are now alive and ready to run
- 3. Both 'return' from fork() — but get DIFFERENT return values
- 4. Which one runs first is decided by the OS scheduler (you cannot predict it)

Return Value Rules (MOST IMPORTANT)

Situation	Return value of fork()
In the PARENT process	PID of the child (positive integer > 0)
In the CHILD process	0 (exactly zero)
fork() FAILED (no child created)	-1 (and errno is set)

Standard fork() Code Pattern

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();           // create child

    if (pid < 0) {                 // fork failed
        perror("fork");
        return 1;
    } else if (pid == 0) {        // I am the CHILD
        printf("Child running, my PID = %d\n", getpid());
        // do child work here
    } else {                     // I am the PARENT (pid = child's PID)
        printf("Parent, child PID = %d\n", pid);
        wait(NULL);              // wait for child to finish
    }
    return 0;
}
```

□ **HOW
TO
READ
THIS**

After `fork()`, there are TWO copies of this program running. Each copy runs the `if-else`. The parent gets `pid>0`, the child gets `pid==0`. They take different branches.

What Gets Copied by `fork()`

Resource	What happens after <code>fork()</code>
Stack memory	Copied — changes in child do NOT affect parent
Heap memory	Copied — changes in child do NOT affect parent
Global/static memory	Copied — changes in child do NOT affect parent
Open files (<code>stdin</code> , <code>stdout</code> , <code>stderr</code>)	SHARED reference — both processes share file position!
Code (text segment)	Shared (read-only), same executable

□ **FILE
SHARING
TRAP**

If both parent and child write to `stdout/stderr`, their output can interleave in unpredictable order. This is a common exam question!

Copy-On-Write (COW)

- Fork does NOT immediately physically copy all memory
- OS marks all pages as shared and read-only initially
- Only when a process modifies a page does OS copy it
- This makes `fork()` fast — only changed pages are duplicated

4. CONCURRENCY AFTER fork()

The Fundamental Rule

❑
**NEVER
ASSUME
ORDER**

After fork(), you CANNOT assume which process (parent or child) runs first. The OS scheduler decides. This is true even on a single-core machine. Both are marked 'ready' simultaneously.

On a multi-core machine, parent and child may literally run at exactly the same time.
On a single-core machine, one gets the CPU first, but you cannot predict which.

Why This Matters for Exams

- If you see a fork() program, the output ORDER can vary — but the content of each process's output is deterministic
- 'Which line prints first?' → Answer: we don't know / either order is possible
- Race conditions happen when code incorrectly assumes ordering

5. WAITING FOR A CHILD: wait() AND waitpid()

Why Wait?

- Parent should collect the exit status of its child
- Without waiting, a terminated child becomes a ZOMBIE process
- wait() blocks (pauses) the parent until a child finishes

wait() and waitpid() Signatures

```
#include <sys/wait.h>

pid_t wait(int *status);

pid_t waitpid(pid_t pid, int *status, int options);

// Pass NULL for status if you don't care about exit code
// wait(NULL) = wait for ANY child to finish
```

What wait() / waitpid() Does

Aspect	Detail
What it does	Blocks the calling (parent) process until a child terminates
Return value	PID of the child that died, or -1 on error

status parameter	Filled with info about HOW the child died (pass NULL to ignore)
waitpid() difference	Can wait for a SPECIFIC child (by PID) and has non-blocking option
Scope	Only waits for DIRECT children — NOT grandchildren

Checking the Exit Status

```
int status;
pid_t dead_child = wait(&status);

if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);    // exit code returned by child
    printf("Child exited normally with code %d\n", code);
}

if (WIFSIGNALED(status)) {
    printf("Child was killed by a signal\n");
}
```

❏ MACROS EXPLAINED

WIFEXITED(status): non-zero if child exited normally. WEXITSTATUS(status): the actual exit code (0–255). WIFSIGNALED(status): non-zero if killed by a signal.

6. ZOMBIES AND ORPHANS

Zombie Process

- A process that has FINISHED running but whose parent has NOT yet called wait()
- It is dead but still occupies an entry in the process table
- You CANNOT kill a zombie (it is already dead)
- It holds almost no resources — just the process table slot
- The zombie is 'reaped' (removed) when the parent calls wait()

ZOMBIE DANGER

If a parent never calls wait() and creates many children, zombie entries accumulate. Eventually the process table fills up and no new processes can be created — a serious system problem.

How Zombies Are Cleared

- Parent calls wait() or waitpid() → zombie reaped immediately
- Parent dies before reaping → 'init' (PID 1) adopts the zombie and reaps it
- Some systems allow parent to declare SIGCHLD = SIG_IGN to auto-reap children

Orphan Process

- A process whose PARENT has died (not the other way around)
- The init process (PID 1) becomes the new parent (adoption)
- Orphans are not harmful — init will wait() for them

Term	What it means
Zombie	Child finished but parent hasn't called wait() yet — process table entry remains
Orphan	Parent died before child — init adopts the child
Reaping	Parent calls wait() to collect child's exit status and free the slot

7. SYSTEM CALLS

What is a System Call?

- An API that a user-space program uses to request services from the OS kernel
- Examples: `fork()`, `read()`, `write()`, `open()`, `exit()`
- More expensive (slower) than regular function calls
- Regular function: stays in user mode. System call: switches to kernel mode and back

How to Look Up System Calls

```
man -S2 intro      # overview of system calls
man -S2 syscalls   # list of all system calls
man -S2 fork       # details about fork()
man -S2 wait       # details about wait()

# Section 2 = system calls
# Section 3 = C library functions
```

System Call	What it does
<code>fork()</code>	Create a child process (clone)
<code>wait()</code> / <code>waitpid()</code>	Parent waits for child to die
<code>exit()</code>	Terminate the current process
<code>getpid()</code>	Return the current process's PID
<code>getppid()</code>	Return the parent's PID
<code>execve()</code>	Replace current process with a new program

8. USEFUL SHELL COMMANDS

Command	What it does and how to use it
ps	List currently running processes. Try: ps aux (all processes with details)
pstree	Show processes as a tree — shows parent-child relationships visually
top	Live dashboard: CPU and memory usage per process. Press q to quit
kill <PID>	Send a signal to process with that PID. Default signal = SIGTERM (polite)
kill -9 <PID>	SIGKILL — forces immediate termination; process cannot ignore this
kill -l	List all available signal names and their numbers
Ctrl-C	Sends SIGINT to the foreground process — usually terminates it

□ kill -9

Signal 9 is SIGKILL. It cannot be caught or ignored by the target process. Use it as a last resort when a normal kill does not work.

9. COMPLETE CODING PATTERNS FOR EXAM

Pattern A: Simple fork with wait

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                // CHILD
        printf("I am the child, PID=%d\n", getpid());
        return 0;                 // child exits
    }
    // PARENT continues here
    wait(NULL);                   // reap child
    printf("Child is done.\n");
    return 0;
}
```

Pattern B: Multiple children

```
int main() {
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {            // each CHILD
            printf("Child %d\n", i);
            return 0;              // IMPORTANT: child must exit here!
        }                         // otherwise child also loops and forks!
    }
    // PARENT waits for all 3 children
    for (int i = 0; i < 3; i++) {
        wait(NULL);
    }
    return 0;
}
```



COMMON MISTAKE

Forgetting return 0 / exit() inside the child branch of a loop. Without it, the child continues the loop and forks more processes — causing a 'fork bomb'!

Pattern C: Getting the child's exit code

```
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        return 42;                // child exits with code 42
    }
    int status;
    waitpid(pid, &status, 0);     // wait for specific child
    if (WIFEXITED(status)) {
        printf("Exit code: %d\n", WEXITSTATUS(status)); // prints 42
    }
}
```

```
    return 0;
}
```

Pattern D: Tracing how many processes exist

Trick: Count the number of times `fork()` is called and check if there's a `return 0` in between.

```
fork();    // now 2 processes
fork();    // each of the 2 forks → now 4 processes
fork();    // each of the 4 forks → now 8 processes

// Rule: N calls to fork() (with no exits) → 2^N processes total
```

□ COUNTING TRICK

With no `exit()` calls in between, each `fork()` doubles the number of processes. 1 fork → 2 processes. 2 forks → 4 processes. 3 forks → 8 processes. Formula: $2^{\text{(number of forks)}}$.

10. MULTIPLE CHOICE QUESTION TIPS

Common MCQ Topics and Quick Answers

Question Type	Key Answer / Rule
What does <code>fork()</code> return in the child?	Exactly 0 (zero)
What does <code>fork()</code> return in the parent?	The PID of the child (positive integer)
What does <code>fork()</code> return on failure?	-1 (no child is created)
What is a zombie process?	A dead process whose parent hasn't called <code>wait()</code> yet
What is a PCB?	Process Control Block — OS record storing all process state
Can you predict which runs first after <code>fork()</code> ?	NO — OS scheduler decides
What happens to memory after <code>fork()</code> ?	Copied; changes in one don't affect the other
What happens to open files after <code>fork()</code> ?	Shared — both processes share the file position
What is the purpose of <code>wait()</code> ?	Block parent until child dies; reap the zombie
What happens if parent dies before child?	<code>init</code> (PID 1) adopts the child
What does <code>kill -9</code> send?	<code>SIGKILL</code> — cannot be caught or ignored
System calls are in which <code>man</code> section?	Section 2 (<code>man -S2</code>)
How many processes after 3 <code>fork()</code> calls?	8 (2^3) — assuming no exits in between
What is time-sharing?	Single CPU rapidly switches between processes

Process State Vocabulary

Term	Definition
Running	Currently executing on the CPU
Ready	Waiting to be given the CPU (in the run queue)
Blocked / Waiting	Waiting for I/O or event — cannot use CPU even if free
Zombie	Finished execution, waiting for parent to collect exit status
Orphan	Parent has died; adopted by <code>init</code>

Tricky Exam Scenarios

SCENARIO 1	Code calls <code>fork()</code> but never calls <code>wait()</code> . The child finishes first. → Child becomes a ZOMBIE until parent calls <code>wait()</code> or parent exits.
SCENARIO 2	Code calls <code>fork()</code> in a loop without <code>exit()</code> in the child branch. → Each child also forks in the loop → EXPONENTIAL process explosion (fork bomb).

**SCENARIO
3**

Parent and child both `printf()` after `fork()`. → Output ORDER is undefined/non-deterministic but both will eventually print.

**SCENARIO
4**

Parent closes a file after `fork()`. → Child still has its own file descriptor open (closing in parent does not affect child).

11. ONE-PAGE SUMMARY CHEATSHEET

CONCEPT	QUICK FACT
Process	Running instance of a program
PCB	OS data structure saving all process state
<code>fork()</code>	Creates a clone of the current process
<code>fork()</code> → parent	Returns child's PID (positive)
<code>fork()</code> → child	Returns 0
<code>fork()</code> → error	Returns -1
<code>wait(NULL)</code>	Block until any child finishes; reap zombie
<code>waitpid(pid, &s, 0)</code>	Wait for specific child
Zombie	Dead child, parent hasn't called <code>wait()</code>
Orphan	Parent died; init adopts child
<code>exit(code)</code>	Terminate process with exit code
<code>getpid()</code>	Returns current process PID
<code>getppid()</code>	Returns parent's PID
<code>kill -9 PID</code>	Force-kill (SIGKILL), cannot be blocked
2^N processes	Result of N <code>fork()</code> calls with no exits
Concurrency	Cannot predict parent vs child order after <code>fork()</code>

Good luck on your exam! ☐